

Q1 Deep Copy of Student Record

Debug a class containing dynamically allocated marks. Implement a deep-copy constructor and destructor. Changing one object's marks must not change the copied object.

```
class Student {  
    int* marks;  
public:  
    Student(int m) { marks = new int(m); }  
    Student(const Student& s) {  
        marks = s.marks; // Bug: shallow copy  
    }  
    void setMarks(int m) { *marks = m; }  
    void display() { cout << *marks << endl; }  
};
```

Your Code

Input

```

1 #include <iostream>
2 using namespace std;
3
4 class Student {
5     int* marks;
6
7 public:
8     Student(int m) {
9         marks = new int(m);
10    }
11
12    // Deep Copy Constructor
13    Student(const Student& s) {
14        marks = new int(*s.marks);
15    }
16
17    void setMarks(int m) {
18        *marks = m;
19    }
20
21    void display() {
22        cout << *marks << endl;
23    }
24
25    // Destructor
26    ~Student() {

```

stdin input

Output

Visible Test Cases

Test Case 1

Q2 Animal Sound Debugging

20 marks

Debug the Animal hierarchy so that Dog, Cat, and Cow override sound(). The correct derived sound must be produced through an Animal pointer.

```
1 #include <iostream>
2 using namespace std;
3
4 class Parent {
5 public:
6     virtual ~Parent() {
7         cout << "Parent Destructor" << endl;
8     }
9 };
10
11 class Child : public Parent {
12 public:
13     ~Child() {
14         cout << "Child Destructor" << endl;
15     }
16 };
17
18 int main() {
19     Parent* p = new Child();
20
21     delete p;
22
23     return 0;
24 }
```

stdin input

Output



Visible Test Cases

Test Case 1

Q3 Bank Account Runtime Polymorphism

20 marks

Write a C++ program using an abstract Account class and derived SavingsAccount, CurrentAccount, and LoanAccount classes. Implement calculateInterest() using runtime polymorphism. Identify and correct the starter-code error so a base pointer invokes the derived implementation.

```

1 #include <iostream>
2 using namespace std;
3
4 class Account {
5 public:
6     virtual void calculateInterest() = 0;
7
8     virtual ~Account() {}
9 };
10
11 class SavingsAccount : public Account {
12     double balance;
13
14 public:
15     SavingsAccount(double b) {
16         balance = b;
17     }
18
19     void calculateInterest() override {
20         cout << "Savings Interest: " << balance * 0.05 << endl;
21     }
22 };
23
24 class CurrentAccount : public Account {
25     double balance;
26
27 public:
28     CurrentAccount(double b) {

```

10000

Output

Visible Test Cases

Test Case 1

Q4 Static Object Counter

Create a static data member in Employee to count all Employee objects created. The count must be shared across objects and accessible without maintaining a separate counter in each object.

```

1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     virtual void displayDetails() {
7         cout << "Vehicle" << endl;
8     }
9
10    virtual ~Vehicle() {}
11 };
12
13 class Car : public Vehicle {
14 public:
15     void displayDetails() override {
16         cout << "Car" << endl;
17     }
18 };
19
20 class Bike : public Vehicle {
21 public:
22     void displayDetails() override {
23         cout << "Bike" << endl;
24     }
25 };

```

stdin input

Output



Visible Test Cases

Test Case 1



Q5 File Manager Exception

20 marks

Create a FileManager class that opens a file and throws an exception if it can't be opened. Handle the error at the application level.

```
class FileManager {  
    public:  
        void openFile(string name) {  
            ifstream file(name);  
            if (!file)  
                throw runtime_error("File cannot be opened");  
        }  
};
```

```

2 using namespace std;
3
4 class MediaPlayer {
5 public:
6     virtual void play() = 0;
7     virtual ~MediaPlayer() {}
8 };
9
10 class MP3Player : public MediaPlayer {
11 public:
12     void play() override {
13         cout << "Playing MP3" << endl;
14     }
15 };
16
17 class WAVPlayer : public MediaPlayer {
18 public:
19     void play() override {
20         cout << "Playing WAV" << endl;
21     }
22 };
23
24 class MP4Player : public MediaPlayer {
25 public:
26     void play() override {
27         cout << "Playing MP4" << endl;
28     }
29 };

```

Output

Visible Test Cases

Test Case 1